

Name: Kuldip Saha Majedha
ID: ITR018

Abstract Algebra and Number Theory Foundations of Cryptography.

1/ Prove Fermat's Little Theorem from group-theoretic first principles, then explain how it motivates the Miller-Rabin probabilistic primality test.

Apply the test (base $a=2$) to $n=561$ and explain why 561 is a ~~Car~~Carmichael number that fools the simpler Fermat test. (6 marks)

Ans:

Fermat's Little Theorem:

If p is a prime number and a is not divisible by p , then $a^{p-1} \equiv 1 \pmod{p}$

Proof:

The non-zero residues $h = \{1, 2, \dots, p-1\}$ form a multiplicative group modulo p , having $p-1$ elements

By Lagrange's theorem, the order of a divides $p-1$. Therefore,

$$a^{p-1} \equiv 1 \pmod{p}$$

Multiplying both sides by a gives,

$$a^p \equiv a \pmod{p}$$

This is Fermat's Little Theorem (FLT).

Motivation for Miller-Rabin test:

FLT says: if p is prime and $\gcd(a, p) = 1$, then $a^{p-1} \equiv 1 \pmod{p}$.

A naive test: check $a^{n-1} \equiv 1 \pmod{n}$. But some composites (Carmichael numbers) also satisfy this for all bases.

Miller-Rabin:

$$n-1 = 2^s d, \quad d \text{ odd.}$$

For prime n , for any base a (coprime to n):

$$a^d \equiv 1 \pmod{n} \text{ or } a^{2^r d} \equiv -1 \pmod{n}$$

for some $0 \leq r < s$

if neither occurs, n is composite. This uses the fact that in a prime modulus, the only square roots of 1 are ± 1 .

Apply to $n=561$, $a=2$:

$$560 = 2^4 \cdot 35 \Rightarrow r=4, d=35$$

$$2^{35} \equiv 263 \pmod{561} \neq \pm 1$$

Squaring,

$$2^{70} = (2^{35})^2 \equiv 166 \pmod{561}$$

$$2^{170} = (2^{70})^2 \equiv 67 \pmod{561}$$

$$2^{380} = (2^{170})^2 \equiv 1 \pmod{561}$$

Since no -1 appears, 561 is composite (fails Miller-Rabin)

Why 561 fools Fermat:

$$561 = 3 \times 11 \times 17$$

It is a Carmichael number, so:

$$a^{560} \equiv 1 \pmod{561} \forall \gcd(a, 561) = 1$$

Thus it passes Fermat test but fails in Miller-Rabin, showing Miller-Rabin is more reliable.

3/ Prove that $\mathbb{Z}$, under addition and multiplication, modulo n , is a field if only if n is prime. Illustrate your proof by contrasting the multiplicative structure of $\mathbb{Z}_{11}$ (Prime modulus) with $\mathbb{Z}_{12}$ (Composite modulus). (5 marks)

Ans:

$\mathbb{Z}_n$ is a field iff n is prime.

□ If field, then n is prime:

Assume $\mathbb{Z}_n$ is a field. Then it has no zero divisors.

If n is composite, $n = ab$ where $1 < a, b < n$.

Then $a \cdot b \equiv 0 \pmod{n}$

Here $a \neq 0, b \neq 0$, but product is 0, that means zero divisor exists.

So, $\mathbb{Z}_n$ cannot be a field.

Hence, n must be prime.

□ If n is prime, then field:

If $n = p$ is prime, then for any $a \neq 0$, $\gcd(a, p) = 1$.

So there exists x such that

$$ax \equiv 1 \pmod{p}$$

So every non-zero element has an inverse. That means it is a field.

Example:

$\mathbb{Z}_{11}$ (prime):

- Every non-zero element has an inverse.

Example:

$$2 \cdot 6 \equiv 1 \pmod{11}$$

Then it's a field.

$\mathbb{Z}_{12}$ (composite):

$$3 \cdot 4 \equiv 0 \pmod{12}$$

- Zero divisors exist;

Then it's not a field.

As a result we can say, $\mathbb{Z}_n$ is a field only when n is prime.

Time (8:26)

3) Construct $GF(2^3)$ using the irreducible polynomial x^3+x+1 over $GF(2)$. Build the multiplication table for a generator element α , and determine whether the multiplicative group $GF(2^3)^*$ is cyclic. (6 marks)

Ans:

Construction of $GF(2^3)$:

Given irreducible polynomial:

$$f(x) = x^3 + x + 1 \text{ over } GF(2)$$

Let α be a root, $f(\alpha) = 0$

$$\alpha^3 + \alpha + 1 = 0$$

$$\Rightarrow \alpha^3 = -\alpha - 1$$

$$\Rightarrow \alpha^3 = \alpha + 1 \quad \left[\begin{array}{l} \text{As } GF(2), \\ -1 = 1, -\alpha = \alpha \end{array} \right]$$

As $GF(2^3)$, $2^3 = 8$ field elements exist.

So, the field elements are:

$$GF(2^3) = \{0, 1, \alpha, \alpha+1, \alpha^2, \alpha^2+1, \alpha^2+\alpha, \alpha^2+\alpha+1\}$$

powers of α :

Using $\alpha^3 = \alpha + 1$:

$$\alpha^1 = \alpha, \alpha^0 = 1$$

$$\alpha^2 = \alpha^2$$

$$\alpha^3 = \alpha + 1$$

$$\alpha^4 = \alpha \cdot \alpha^3 = \alpha(\alpha + 1) = \alpha^2 + \alpha$$

$$\begin{aligned}\alpha^5 &= \alpha \cdot \alpha^4 = \alpha(\alpha^2 + \alpha) = \alpha^3 + \alpha^2 \\ &= (\alpha + 1) + \alpha^2 = \alpha^2 + \alpha + 1\end{aligned}$$

$$\begin{aligned}\alpha^6 &= \alpha \cdot \alpha^5 = \alpha(\alpha^2 + \alpha + 1) = \alpha^3 + \alpha^2 + \alpha \\ &= (\alpha + 1) + \alpha^2 + \alpha = \alpha^2 + \cancel{\alpha + \alpha} + 1 + \alpha + \alpha \\ &= \alpha^2 + 1 \quad \left[\begin{array}{l} \text{at GF(2) rule,} \\ \alpha + \alpha = 0 \end{array} \right]\end{aligned}$$

$$\begin{aligned}\alpha^7 &= \alpha \cdot \alpha^6 = \alpha(\alpha^2 + 1) = \alpha^3 + \alpha \\ &= (\alpha + 1) + \alpha \\ &= 1 \quad [\alpha + \alpha = 0]\end{aligned}$$

Multiplication Table for $\text{GF}(2^3)^*$:

The group contains $2^3 - 1 = 7$ non zero elements: $\{\alpha^0, \alpha^1, \alpha^2, \alpha^3, \alpha^4, \alpha^5, \alpha^6\}$.
 Multiplication follows exponent addition, modulo 7
 ($\alpha^i \cdot \alpha^j = \alpha^{(i+j) \bmod 7}$):

$$\alpha \quad \quad \quad 1 \quad \quad \quad \alpha \quad \quad \quad \alpha^2$$

x	1	α	α^2	α^3	α^4	α^5	α^6
1	1	α	α^2	α^3	α^4	α^5	α^6
α	α	α^2	α^3	α^4	α^5	α^6	1
α^2	α^2	α^3	α^4	α^5	α^6	1	α
α^3	α^3	α^4	α^5	α^6	1	α	α^2
α^4	α^4	α^5	α^6	1	α	α^2	α^3
α^5	α^5	α^6	1	α	α^2	α^3	α^4
α^6	α^6	1	α	α^2	α^3	α^4	α^5

Cyclicity of $GF(2^3)^*$:

Group order: $|GF(2^3)^*| = 2^3 - 1 = 7$ (a prime order).

order of α :

$$\alpha^1 \neq 1, \alpha^2 \neq 1, \alpha^3 \neq 1, \alpha^4 \neq 1, \alpha^5 \neq 1, \alpha^6 \neq 1, \alpha^7 = 1$$

$$\therefore \text{ord}(\alpha) = 7$$

Since, $\text{ord}(\alpha) = |GF(2^3)^*| = 7$, the single element α generates the entire non-zero multiplicative group:

$$GF(2^3)^* = \langle \alpha \rangle = \{ \alpha^0, \alpha^1, \alpha^2, \alpha^3, \alpha^4, \alpha^5, \alpha^6 \}$$

Therefore, the multiplicative group $GF(2^3)^*$ is cyclic, and α is a primitive element.

Time (17:51)

4/ Derive the time complexity of modular exponentiation via square-and-multiply. Compute $7^{560} \pmod{561}$ step by step and interpret the result in light of 561 being a Carmichael number. (5 marks).

Ans:

In square and multiply, the exponent e is exposed bit by bit or processed bit by bit. For each bit -

- 1 squaring is performed
- If the bit is 1, 1 multiplication is performed.

The number of bits in e is: $\lceil \log_2 e \rceil + 1$

$\therefore$ The time complexity is: $O(\log e)$

Compute $7^{560} \pmod{561}$:

$$560 = 512 + 32 + 16 = 2^9 + 2^5 + 2^4$$

We need 7^{512} , 7^{32} , 7^{16}

Successive squaring modulo 561:

$$7^2 \equiv 49 \pmod{561}$$

$$7^4 \equiv 157 \pmod{561} = 49^2 = 2401$$

$$7^8 \equiv 526 \pmod{561} = 157^2 = 24649 \\ \equiv -35 \pmod{561}$$

$$7^{16} \equiv (-35)^2 = 1225 \equiv 103 \pmod{561}$$

$$7^{32} \equiv (103)^2 = 10609 \equiv 511 \pmod{561} \\ \equiv -50 \pmod{561}$$

$$7^{64} \equiv (-50)^2 = 2500 \equiv 256 \pmod{561}$$

$$7^{128} \equiv (256)^2 = 65536 \equiv 460 \pmod{561} \\ \equiv -101 \pmod{561}$$

$$7^{256} \equiv (-101)^2 = 10201 \equiv 103 \pmod{561}$$

$$7^{512} \equiv (103)^2 \equiv -50 \pmod{561}$$

Combining,

$$7^{560} = 7^{512} \cdot 7^{32} \cdot 7^{16} \pmod{561}$$

$$\cdot 7^{512} \cdot 7^{32} \equiv (-50)(-50) = 2500 \equiv 256 \pmod{561}$$

$$\cdot 256 \cdot 7^{16} \equiv 256 \times 103 = 26368 \equiv 1 \pmod{561}$$

$$\therefore 7^{560} \equiv 1 \pmod{561}$$

Interpretation (Carmichael Number):

- 561 is composite ($561 = 3 \times 11 \times 17$), yet it satisfies $7^{560} \equiv 1 \pmod{561}$.
- By Korselt's Criterion, $(p-1) \mid 560$ for all factors $p \in \{3, 11, 17\}$, making 561 a Carmichael number.

where $a^{561} \equiv 1 \pmod{561}$ for all $\gcd(a, 561) = 1$.

This shows the Fermat test fails on Carmichael numbers, necessitating the Miller-Rabin test in cryptography. time (17:8)

5/ Compare deterministic (AKS) and probabilistic (Miller-Rabin) primality testing in terms of asymptotic complexity, error probability and practical relevance to RSA key generation at 2048 bit and 4096 bit key sizes. (9-Marks)

Ans:

Comparison between AKS vs Miller-Rabin:

Metric	AKS Algorithm	Miller-Rabin Algorithm
Type	Deterministic (100% unconditional)	Probabilistic (Monte Carlo)
Bit Complexity	$\tilde{O}((\log n)^6)$	$O(k \cdot (\log n)^3)$ (k rounds)
Error	0 (No false results)	$\leq 4^{-k} = 2^{-2k}$
practical speed	Extremely slow (impractical)	Fast (milliseconds)

Error Probability Analysis:

For any odd composite n , each independent random base a has an error probability $\leq \frac{1}{4}$.

After k independent rounds:

$$P(\text{Composite passes as prime}) \leq \left(\frac{1}{4}\right)^k = 2^{-2k}$$

For $k=64$,

$$P(\text{Error}) \leq 2^{-128}$$

This is negligible and far below standard hardware failure rates.

Relevance to RSA-2048 and RSA-4096 Key Generation

The problem: Generating an RSA key requires finding two large ~~number~~ primes (p, q) by testing thousands of random candidates.

Why AKS fails in practice: Its $\approx (\log n)^6$ complexity causes an acceptable latency (hours/days per key) for 1024-bit to 2048-bit primes.

Industry Standard: Production libraries (OpenSSL, BouncyCastle) exclusively pair small-prime trial division with Miller-Rabin ($k=40$ to 64) to confirm primality in milliseconds.

Modular Arithmetic & Chinese Remainder Theorem

Derive the Extended Euclidean Algorithm formally (as a sequence of linear combinations), then use it to compute $\gcd(290, 96)$ together with the Bézout coefficients x, y such that $290x + 96y = \gcd(290, 96)$ (5 marks)

Ans:

Derivation of Extended Euclidean Algorithm:

The Extended Euclidean Algorithm computes $\gcd(a, b)$ along with coefficients x, y such that:

$$ax + by = \gcd(a, b)$$

using standard Euclidean division steps:

1. $r_{i-2} = q_i \cdot r_{i-1} + r_i \quad (0 \leq r_i < r_{i-1})$

2. Express each remainder as a linear combination of a and b :

$$r_i = r_{i-2} - q_i \cdot r_{i-1}$$

3. Working backwards (back-substitution) from the last non-zero remainder up to the first equation substitutes intermediate remainders, yielding the Bézout coefficients x and y .

Computation of $\gcd(240, 46)$ and Bézout Coefficients:

Step 1: Forward Division (Euclidean Algorithm):

$$240 = 5 \times 46 + 10 \Rightarrow 10 = 240 - 5 \times 46$$

$$46 = 4 \times 10 + 6 \Rightarrow 6 = 46 - 4 \times 10$$

$$10 = 1 \times 6 + 4 \Rightarrow 4 = 10 - 1 \times 6$$

$$6 = 1 \times 4 + 2 \Rightarrow 2 = 6 - 1 \times 4$$

$$4 = 2 \times 2 + 0 \Rightarrow \gcd(240, 46) = 2$$

Step 2: Backward Substitution (Extended EA):

Express the last non-zero remainder (2) working upwards:

$$2 = 6 - 1 \times 4$$

$$\text{Substitute } 4 = 10 - 1 \times 6$$

$$\begin{aligned} \therefore 2 &= 6 - 1 \times (10 - 1 \times 6) = 6 - 10 + 1 \times 6 \\ &= 2 \times 6 - 1 \times 10 \end{aligned}$$

$$\text{Substitute } 6 = 46 - 4 \times 10$$

$$\begin{aligned} \therefore 2 &= 2 \times (46 - 4 \times 10) - 1 \times 10 \\ &= 2 \times 46 - (2 \times 4) \times 10 - 1 \times 10 \\ &= 2 \times 46 - 8 \times 10 - 1 \times 10 \\ &= 2 \times 46 - 9 \times 10 \end{aligned}$$

$$\text{Substitute } 10 = 240 - 5 \times 46$$

$$\therefore 2 = 2 \times 46 - 9 \times (240 - 5 \times 46)$$

$$= 2 \times 46 - 9 \times 290 + (-9 \times -5) \times 46$$

$$= 2 \times 46 - 9 \times 290 + 45 \times 46$$

$$= -9 \times 290 + 47 \times 46$$

Final result:

$$\gcd(290, 46) = 2$$

$$x = -9, y = 47 \Rightarrow 290(-9) + 46(47) = 2$$

Time (16:09)

Q State and prove the Chinese Remainder Theorem (existence and uniqueness) for pairwise coprime moduli. Apply it to solve: (6 marks)

$$x \equiv 2 \pmod{3}, x \equiv 3 \pmod{5}, x \equiv 4 \pmod{7}, x \equiv 5 \pmod{11}$$

Ans:

Statement & Proof of Chinese Remainder Theorem (CRT):

Theorem: If $m_1, m_2, \dots, m_k$ are pairwise coprime positive integers, then the system $x \equiv a_i \pmod{m_i}$ ($1 \leq i \leq k$) has a unique solution modulo $M = \prod_{i=1}^k m_i$.

Proof:

Existence:

- Let $M = \prod_{i=1}^k m_i$ and $M_i = M/m_i$.
- Since $\gcd(M_i, m_i) = 1$, an inverse y_i exists such that $M_i y_i \equiv 1 \pmod{m_i}$.
- Construct $x = \sum_{i=1}^k a_i M_i y_i$.

• for any i : $M_i \equiv 0 \pmod{m_i}$ when $i \neq j$, and $M_i y_i \equiv 1 \pmod{m_i}$.

• Hence, $x \equiv a_j(1) \equiv a_j \pmod{m_j}$ for all i .

Q Uniqueness:

• If x_1, x_2 are solutions, $m_i | (x_1 - x_2)$ for all i .

Since moduli are pairwise coprime, $M | (x_1 - x_2)$
 $\Rightarrow x_1 \equiv x_2 \pmod{M}$.

Numerical Solution:

Given,

$$x \equiv 2 \pmod{3}, x \equiv 3 \pmod{5}, x \equiv 4 \pmod{7}, x \equiv 5 \pmod{11}$$

$$\text{Modulo Product, } M = 3 \times 5 \times 7 \times 11 = 1155$$

Compute M_i and Inverses $y_i \equiv M_i^{-1} \pmod{m_i}$:

$$M_1 = \frac{1155}{3} = 385 \equiv 1 \pmod{3}$$

$$\Rightarrow 1 \cdot y_1 \equiv 1 \Rightarrow y_1 = 1$$

$$M_2 = \frac{1155}{5} = 231 \equiv 1 \pmod{5}$$

$$\Rightarrow 1 \cdot y_2 \equiv 1 \Rightarrow y_2 = 1$$

$$M_3 = \frac{1155}{7} = 165 \equiv 4 \pmod{7}$$

$$\Rightarrow 4 \cdot y_3 \equiv 1 \Rightarrow y_3 = 2 \quad (4 \times 2 = 8 \equiv 1)$$

$$M_4 = \frac{1155}{11} = 105 \equiv 6 \pmod{11}$$

$$\Rightarrow 6 \cdot y_4 \equiv 1 \Rightarrow y_4 = 2 \quad (6 \times 2 = 12 \equiv 1)$$

Answer side solution:

$$x = \sum a_i m_i y_i = [2(385)(1) + 3(231)(1) + 4(165)(2) + 5(105)(2)]$$

$$= 3833$$

$$\therefore x = 3833 \pmod{1155}$$

$$= 368 \pmod{1155}$$

Time (19:22)

8/ Derive the CRT-based RSA decryption/signing optimization from first principles (computing d_p , d_q and q_{inv}) and demonstrate the ~~1-4x~~ ~1x speed up with a fully worked numerical example using small prime p, q . (6-marks)

Ans:

Derivation of CRT parameters:

In RSA, standard decryption is $M \equiv C^d \pmod{n}$ where $n = p \cdot q$. Using CRT:

- Exponents (d_p, d_q): By Fermat's Little Theorem, exponents reduce modulo $(p-1)$ and $(q-1)$:

$$d_p = d \pmod{p-1}, \quad d_q = d \pmod{q-1}$$

$$m_1 \equiv C^{d_r} \pmod{p}, \quad m_2 \equiv C^{d_a} \pmod{q}$$

- Recombination (G Garner's Formula):

$$h = q_{inv} \cdot (m_1 - m_2) \pmod{p}, \text{ where } q_{inv} = q^{-1} \pmod{p}$$

$$M = m_2 + h \cdot q$$

$\approx 4\times$ Speed-up Derivation:

Let n have k bits; primes p, q have $\approx k/2$ bits:

- standard ($C^d \pmod{n}$): Time complexity = $O(k^3)$

- CRT method:

- Computing modulo p : $O((k/2)^3) = \frac{1}{8} O(k^3)$

- Computing modulo q : $O((k/2)^3) = \frac{1}{8} O(k^3)$

- Total time = $2 \times \frac{1}{8} O(k^3)$
 $= \frac{1}{4} O(k^3)$

$$\text{Speed-up} \approx \frac{O(k^3)}{\frac{1}{4} O(k^3)} = 4\times$$

Numerical Example:

Setup: Primes $p=5, q=11 \Rightarrow n=55, \phi(n) = 40$.

public key $e=3 \Rightarrow d=27$ where $ed \equiv 1 \pmod{\phi(n)}$

Ciphertext $C=19$.

Pre Computations:

- $d_p = 27 \bmod 4 = 3 = 27 \bmod (p-1)$

- $d_q = 27 \bmod 10 = 7 = 27 \bmod (q-1)$

- $q_{inv} = 11^{-1} \bmod 5 = 1^{-1} \bmod 5 = 1$

Modular Exponentiations:

- $m_1 = 14^3 \equiv \cancel{4^3} \equiv 4 \pmod{5} = c^{d_p} \pmod{p}$
 $\equiv (14 \equiv 4 \pmod{5}) \Rightarrow 4^3 = 64 \equiv 4 \pmod{5})$

- $m_2 = 14^7 \equiv 9 \pmod{11} = c^{d_q} \pmod{q}$

$$(14 \equiv 3 \pmod{11}) \Rightarrow 3^7 = 2187 \equiv 9 \pmod{11})$$

Reconstruction:

- $h = q_{inv} \cdot (m_1 - m_2) \bmod p$

$$= 1 \cdot (4 - 9) \bmod 5 = -5 \equiv 0 \pmod{5}$$

$$\therefore m = 9 + 0 \cdot 11 = 9$$

for verification,

$$14^{27} \pmod{55} = 9$$

Time (29:03)

9) Explain the Bellare fault-injection attack on RSA-CRT signatures: how a single faulty modular computation during signing leaks a factor of n , and what countermeasures (e.g. signature verification before release) prevent it. (5 marks)

Ans:

Attack Mechanism (Fault Injection):

In RSA-CRT signing, two components are computed:

$$S_1 \equiv m^{d_1} \pmod{p}, \quad S_2 \equiv m^{d_2} \pmod{q}$$

- Induced Fault: A hardware glitch corrupts S_1 into $\tilde{S}_1 \neq S_1$, while S_2 remains correct.

- Faulty Signature ($\tilde{S}$):

CRT recombination yields $\tilde{S}$ such that:

$$\tilde{S}^e \not\equiv m \pmod{p} \Rightarrow p \nmid (\tilde{S}^e - m)$$

$$\tilde{S}^e \equiv m \pmod{q} \Rightarrow q \mid (\tilde{S}^e - m)$$

- Factor Recovery: Because $(\tilde{S}^e - m)$ is a multiple of q but not p , a single GCD with public modulus $n = pq$ extracts the private prime factor:

$$q = \gcd(\tilde{S}^e - m, n)$$

Countermeasures:

- Signature Verification Before Release: Signer checks $S^e \equiv m \pmod{n}$ using public exponent e before transmitting. If incorrect, suppress output.
- Cryptographic Blinding: Multiply m by a secret random mask $r \pmod{n}$ prior to CRT signing and remove r afterward, breaking the algebraic relation needed for the GCD attack.
- Redundant Computation: Compute the signature twice across independent cores and verify consistency before output. *Tip (2:11)*

10 Compute a modular inverse using both the Extended Euclidean Algorithm and Fermat's Little Theorem based exponentiation (for a prime modulus) and discuss the trade offs between the two approaches for large moduli. (4 Marks)

Ans:

Modular Inverse Computation ($5x \equiv 1 \pmod{17}$):

- Extended Euclidean Algorithm (EEA):

$$17 = 3 \times 5 + 2 \Rightarrow 2 = 17 - 3 \times 5$$

$$5 = 2 \times 2 + 1 \Rightarrow 1 = 5 - 2 \times 2$$

$$= 5 - 2 \times (17 - 3 \times 5)$$

$$= 5 - 2(17) + 6 \times 5$$

$$= 7 \times 5 - 2 \times 17$$

Reducing modulo 17:

$$5x \equiv 1 \pmod{17}$$

$$\Rightarrow 5 \times 7 \equiv 1 \pmod{17}$$

$$\Rightarrow 5^{-1} \equiv 7 \pmod{17}$$

- Fermat's Little Theorem (FLT):

$$a^{-1} \equiv a^{p-2} \pmod{p}$$

$$\Rightarrow 5^{-1} \equiv 5^{15} \pmod{17}$$

$$\bullet 5^2 = 25 \equiv 8 \pmod{17}$$

$$\bullet 5^4 \equiv 8^2 = 64 \equiv -4 \pmod{17}$$

$$\bullet 5^8 \equiv (-4)^2 = 16 \equiv -1 \pmod{17}$$

$$\therefore 5^{15} = 5^8 \cdot 5^4 \cdot 5^2 \cdot 5$$

$$= (-1)(-4) \cdot 8 \cdot 5 = 160 \equiv 7 \pmod{17}$$

Trade-offs for large Modulus:

- **Computation Complexity:** EEA runs in $O((\log p)^2)$ bit operations versus $O((\log p)^3)$ for FLT square-and-multiply. EEA is significantly faster for large integers.
- **Generality:** EEA works for any modulus m with $\gcd(a, m) = 1$, whereas FLT strictly requires a prime modulus.
- **Side channel - Security:** EEA has input-dependent branching (vulnerable to timing attack). FLT has a fixed execution path, making constant-time implementation much easier in secure hardware (e.g. smart cards).

Time(12:14)

Q Classical & Symmetric Block Ciphers

11) Design and fully execute a Playfair Cipher encryption and decryption of a phrase of your choosing (minimum 26 Letters) with a key word of your choosing. present the complete 5×5 key matrix as a table, and show every digraph substitution step. (6 marks).

Ans:

KEY word: MONARCHY

Plaintext: INSTRUMENTSARESAFE

Diagrams:

I N S T R U M E N T S A R E S A F E

Key matrix (5x5):

M	O	N	A	R
C	H	/	B	D
E	F	G	I/J	K
L	A P	Q	S	T
U	V	W	X	Z

Encryption (Playfair rules):

IN $\rightarrow$ GA

RE $\rightarrow$ MK

ST $\rightarrow$ TL

SA $\rightarrow$ XB

RU $\rightarrow$ MZ

FE $\rightarrow$ GF

ME $\rightarrow$ CL

NT $\rightarrow$ RQ

SA $\rightarrow$ XB

∴ Cipher text: GATLMZCLRQXBMKXBGF

Decryption (reverse rules):

Now plain text: GATLMZCLRQXBMKXBGF

Digraphs: GA TL MZ CL RQ XB MK XB GF

GA → IN

MK → RE

TL → ST

XB → SA

MZ → RU

GF → FE

CL → ME

RQ → NT

XB → SA

Recovered Plaintext:

INSTRUMENTSARESAFE

Playfair cipher uses a 5x5 matrix and digraph substitution. Encryption and decryption successfully recover the original message. Time (14:01)

121 Critically analyze the structural weakness of the Playfair cipher against digraph frequency analysis and explain why it remains vulnerable to known-plaintext attacks despite being twice stronger than monoalphabetic substitution. (5 marks)

Weaknesses Against Digraph Frequency Analysis:

- Small State Space (600 ~~And~~ Digraphs): A 5×5 matrix allows only $25 \times 24 = 600$ valid digraphs. This space is small enough that high-frequency English pairs (TH, HE, IN, ER) directly map to high-frequency ciphertext digraphs.
- Geometric Invariants:
 - Symmetry: If $AB \rightarrow CD$ via the rectangle rule, then $BA \rightarrow DC$.
 - Alignment Leaks: Shared letters between pairs (e.g., $AB \rightarrow CD$ and $EB \rightarrow FD$) reveal the inputs share rows/columns.
- Preserved Language Redundancy: Playfair ~~flattens~~ flattens single-letter frequencies but preserves digraph distribution, enabling statistical reconstruction from a few hundred ciphertext words.

Vulnerability to Known-Plaintext Attacks (KPA):

- Matrix Coordinate Equations: Each known pair $(P_1 P_2 \leftrightarrow C_1 C_2)$ defines exact row/column constraints in the 5×5 grid (e.g., P_1 and C_1 share a row; P_1 and C_2 share a column).

- Coupled Interdependence: Unlike monoalphabetic ciphers (where all 26 letters must be solved independently), playfair letters are geometrically coupled.
- Complete Key Recovery: Only 10-15 known digraph pairs provide enough intersecting row/column constraints to recover the keyword and reconstruct the entire 25-cell key matrix. (15:23) Time

13] Derive and trace two full rounds of a simplified Feistel-structured cipher (DES-like), showing $L_0, R_0 \rightarrow L_2, R_2$ symbolically, then with the numeric values $L_0 = 0xAAAAAAAA, R_0 = 0xF0F0F0F0, K_1 = 0x0F0F0F0F, K_2 = 0x33333333$, using XOR as the round function. (6 marks)

Ans:

Symbolic Derivation of Two Feistel Rounds:

Given an input block split into (L_0, R_0) and round keys K_1, K_2 with round function

$$F(R, k) = R \oplus k :$$

Round 1:

$$L_1 = R_0$$

$$R_1 = L_0 \oplus F(R_0, K_1) = L_0 \oplus (R_0 \oplus K_1)$$

Round 2:

$$L_2 = R_1$$

$$R_2 = L_1 \oplus F(R_1, K_2) = L_1 \oplus (R_1 \oplus K_2)$$

Numeric Trace:

Initial values:

$$L_0 = 0x\text{AAAAAAAA} = (1010\ 1010\ 1010\ 1010\ 1010\ 1010\ 1010\ 1010)_2$$

$$R_0 = 0x\text{FOFOFOFO} = (1111\ 0000\ 1111\ 0000\ 1111\ 0000\ 1111\ 0000)_2$$

$$K_1 = 0x\text{0F0F0F0F} = (0000\ 1111\ 0000\ 1111\ 0000\ 1111\ 0000\ 1111)_2$$

$$K_2 = 0x\text{33333333} = (0011\ 0011\ 0011\ 0011\ 0011\ 0011\ 0011\ 0011)_2$$

Round 1 Execution:

1. Compute round function:

$$\begin{aligned} F(R_0, K_1) &= R_0 \oplus K_1 = 0x\text{FOFOFOFO} \oplus 0x\text{0F0F0F0F} \\ &= 0x\text{FFFFFFF} \end{aligned}$$

2. Update halves:

$$L_1 = R_0 = 0x\text{FOFOFOFO}$$

$$\begin{aligned} \therefore R_1 &= L_0 \oplus F(R_0, K_1) = 0x\text{AAAAAAAA} \oplus 0x\text{FFFFFFF} \\ &= 0x\text{55555555} \end{aligned}$$

Round 2 Execution:

1. Compute round function:

$$\begin{aligned} F(R_1, K_2) &= R_1 \oplus K_2 = 0x\text{55555555} \oplus 0x\text{33333333} \\ &= 0x\text{66666666} \end{aligned}$$

2. Update halves:

$$L_2 = R_1 = 0x55555555$$

$$R_2 = L_1 \oplus F(R_1, k_2) = 0xFOFOFOFO \oplus 0x36666666 \\ = 0x96969696$$

$\therefore$ Final output state:

$$(L_2, R_2) = (0x55555555, 0x96969696)$$

Time (18:49)

14/ Formally define the avalanche effect in block ciphers. Using a toy 8-bit substitution-permutation network of your own construction, demonstrate how flipping a single input bit changes at least half the output bits, and explain why this property is essential for cryptographic security. (5-marks)

Ans:

Formal Definition:

The Avalanche Effect (Strict Avalanche Criterion) requires that flipping a single bit in the plaintext or key causes each ciphertext bit to invert with independent probability $p \approx 0.5$. For an n -bit block, on average of $n/2$ bits must flip.

Toy 8-bit SPN Demonstration:

Design:

- 8-bit block split into two 4-bit nibbles $[N_1, N_0]$
- S-box: $S(0) = C_{16} = 1100_2$, $S(1) = 5_{16} = 0101_2$,
 $S(3) = B_{16} = 1011_2$, $S(15) = 2_{16} = 0010_2$, $S(11) = 8_{16} = 1000_2$
- P-box: Interleaves bits $[0, 1, 2, 3, 4, 5, 6, 7]$
 $\Rightarrow [0, 2, 4, 6, 1, 3, 5, 7]$

Trace (1-bit flip at LSB):

- Plaintexts: $P_1 = 0x00 = (0000\ 0000)_2$ and
 $P_2 = 0x01 = (0000\ 0001)_2$
- Round 1 (S-Box Confusion):
 - $P_1 \rightarrow [S(0), S(0)] = 0xC$
 - $P_2 \rightarrow [S(0), S(1)] = 0xC5$
- Round 1 (P-Box Diffusion):
 - $0xC \rightarrow 0xF0 = [15, 0]$
 - $0xC5 \rightarrow 0xB3 = [11, 3]$
- Round 2 (S-Box):
 - $C_1 = [S(15), S(0)] = 0x2C = (0010\ 1100)_2$
 - $C_2 = [S(11), S(3)] = 0x85 = (1000\ 0101)_2$
- Output Difference:
 $\Delta C = 0x2C \oplus 0x85 = (1010\ 1001)_2$
 $\Rightarrow 4$ out of 8 bits flipped (50%)

Cryptographic Importance:

- Thwarts Differential Cryptanalysis: prevents attackers from tracing predictable difference propagation trails through rounds.
- Destroys statistical patterns: Ensures ciphertext behaves like pseudorandom white noise with zero correlation to plaintext. (17:01) Time

15] Evaluate the brute-force resistance of an 8-character alphanumeric password (62-character alphabet) versus a 6-digit PIN, under a modern GPU ~~crack~~ cracking rate 10^9 guesses/second. Compute worst case and expected case cracking times for both and discuss the implications for password policy design. (9 marks).

Ans:

Cracking Time Computations (Rate $R = 10^9$ guesses/s):

6-Digit PIN ($N=10, L=6$):

- Space, $S = 10^6 = 1000000$
- Worst Case, $T_{wc} = \frac{10^6}{10^9} = 1 \text{ ms}$
- Expected Case, $T_{exp} = \frac{T_{wc}}{2} = 0.5 \text{ ms}$

8-Char Alphanumeric Password ($N=62, L=8$):

- Space, $S = 62^8 \approx 2.183 \times 10^{14}$
- Worst Case, $T_{wc} = \frac{2.183 \times 10^{14}}{10^9} \approx 218340 \text{ s}$
 $\approx 60.65 \text{ hours} \approx 2.53 \text{ days}$

• Expected Case: $T_{exp} = \frac{T_{wc}}{2} \approx 1091700$

$\approx 30.33 \text{ hours} \approx 1.26 \text{ days}$

Policy Implications:

- PINs Require Strict Throttling: Cracking takes under 1mn offline; PINs are only secure with hardware lockout after 3-5 failed tries.
 - 8 characters is Insufficient: ≈ 30 hours to crack means policy must require $\geq 12-16$ characters (or passphrases). Length matters far more than complexity.
 - Slow Hashing is Essential: Systems must use memory-hard KDFs (Argon2id, bcrypt) to throttle GPU guess rates down from $10^9/s$ to $< 10^3/s$. (9:10) time
- ④ Inside AES: Modes, Transformations & Side channels

16

Block Cipher Mode Diagrams:

- ECB: Independent blocks

$$C_i = E_K(P_i), P_i = D_K(C_i)$$

- CBC: Chained through previous ciphertext ($C_0 = IV$).

$$C_i = E_K(P_i \oplus C_{i-1}), P_i = D_K(C_i) \oplus C_{i-1}$$

- CFB: Stream mode from cipher feedback ($C_0 = IV$).

$$C_i = P_i \oplus E_K(C_{i-1}), P_i = C_i \oplus E_K(C_{i-1})$$

• OFB: Internal key stream generation ($O_0 = IV$):

$$O_i = E_K(O_{i-1}), C_i = P_i \oplus O_i, P_i = C_i \oplus O_i$$

• CTR: Counter-driven key stream.

$$C_i = P_i \oplus E_K(\text{Nonce} \parallel i), P_i = C_i \oplus E_K(\text{Nonce} \parallel i)$$

Comparison table:

Mode	Parallel Encryption	Parallel Decryption	Error Propagation (1-bit flip in C_i)	Unique IV, Nonce
ECB	Yes	Yes	Corrupts only corresponding plaintext block	No
CBC	No	Yes	1 bit flip in P_{i+1} ; Completely corrupts P_i	Yes
CFB	No	Yes	1 bit flip in P_i ; Completely corrupts P_{i+1}	Yes
OFB	No	No	Single bit only (flips exact matching bit in P_i)	Yes
CTR	Yes	Yes	Single bit only (flips exact matching bit P_i)	Yes

Recommendations & Justifications:

a) Live video over long network $\rightarrow$ CTR Mode:

- No error propagation: A corrupted ciphertext bit flips only that exact bit in video data; it never ruins adjacent bytes or frames.

- Low latency: key stream blocks can be pre-computed, reducing packet processing to a single fast XOR.

b) Static Disk Image $\rightarrow$ CBE mode:

- Pattern Hiding: CBE with randomized IVs prevents identical disk sectors from generating identical ciphertext.
- Fast Reads: Supports fully parallel decryption across multiple CPU cores for high random-read throughput. Time (16:13)

17 SubByte Transformation:

The input state word is:

$$W = [0x23, 0xA7, 0x4C, 0x19]$$

In AES, each byte $0xRC$ is replaced by indexing the S-box at row R (high nibble) and Column C (low nibble).

Case A: Using the provided Partial S-Box Lookup Table

- $0x23 \rightarrow (\text{Row } 2, \text{Col } 3) = 0x16$
- $0xA7 \rightarrow (\text{Row } A, \text{Col } 7) = 0x63$
- $0x4C \rightarrow (\text{Row } 4, \text{Col } C) = 0x12$
- $0x19 \rightarrow (\text{Row } 1, \text{Col } 9) = 0xD9$

- Resulting transformed word:

$$w' = [0x16, 0x63, 0x12, 0xD9]$$

State Matrix Setup & Shift Rows Transformation:

place the transformed word as the first row (Row 0) and fill rows 1, 2 and 3 with 0x00;

$$S = \begin{bmatrix} 0x16 & 0x63 & 0x12 & 0xD9 \\ 0x00 & 0x00 & 0x00 & 0x00 \\ 0x00 & 0x00 & 0x00 & 0x00 \\ 0x00 & 0x00 & 0x00 & 0x00 \end{bmatrix}$$

- Shift Rows Definition:

- Row 0: Left circular shift by 0 bytes

$$\Rightarrow [0x16, 0x63, 0x12, 0xD9]$$

- Row 1: Left circular shift by 1 bytes

$$\Rightarrow [0x00, 0x00, 0x00, 0x00]$$

- Row 2: Left circular shift by 2 bytes

$$\Rightarrow [0x00, 0x00, 0x00, 0x00]$$

- Row 3: Left circular shift by 3 bytes

$$\Rightarrow [0x00, 0x00, 0x00, 0x00]$$

Resulting Output State Matrix:

$$S_{\text{Shift Rows}} = \begin{bmatrix} 0x16 & 0x63 & 0x12 & 0xD9 \\ 0x00 & 0x00 & 0x00 & 0x00 \\ 0x00 & 0x00 & 0x00 & 0x00 \\ 0x00 & 0x00 & 0x00 & 0x00 \end{bmatrix}$$

18. AES-128 Key Expansion Rule:

A 16-byte key $K_0 = \{w_0, w_1, w_2, w_3\}$ expands into 44 words (w_0 to w_{43}):

$$w_i = \begin{cases} w_{i-4} \oplus g(w_{i-3}) & \text{if } i \equiv 0 \pmod{4} \\ w_{i-1} \oplus w_{i-2} & \text{if } i \not\equiv 0 \pmod{4} \end{cases}$$

• Core $g(w)$ steps:

1. RotWord: Circular left-shift 1 byte $\rightarrow [b_1, b_2, b_3, b_0]$
2. SubWord: S-box each byte $\rightarrow [S(b_1), S(b_2), S(b_3), S(b_0)]$
3. Rcon XOR: $\oplus [RC[i], 00, 00, 00]$ ($RC[1] = 0x01$, $RC[2] = 0x02$)

Computation of K_1 and K_2 (key: All $0x00$):

Base key: $w_0 = w_1 = w_2 = w_3 = [00, 00, 00, 00]$.

(S-box values: $S(0x0) = 63$, $S(0x2) = AA$, $S(0x3) = FB$)

A. Round key K_1 ($w_4, \dots, w_7$):

$g(w_3)$:

• RotWord (w_3) = $[00, 00, 00, 00]$

• SubWord $\hat{=}$ $[63, 63, 63, 63]$

• Rcon[1] $\oplus$ = $[63 \oplus 01, 63, 63, 63] = [62, 63, 63, 63]$

words:

• $w_4 = w_0 \oplus g(w_3) = [62, 63, 63, 63]$

• $w_5 = w_1 \oplus w_4 = [62, 63, 63, 63]$

• $w_6 = w_2 \oplus w_5 = w_7 = w_3 \oplus w_6 = [62, 63, 63, 63]$

$$\therefore K_1 = \begin{bmatrix} 62 & 62 & 62 & 62 \\ 63 & 63 & 63 & 63 \\ 63 & 63 & 63 & 63 \\ 63 & 63 & 63 & 63 \end{bmatrix}$$

B. Round key K_2 ($W_8 \dots W_{11}$):

$g(W_7)$:

- RotWord(W_7) = $[63, 63, 63, 69]$

- SubWord = $[FB, FB, FB, AA]$

- Rcon $^{[2]} \oplus = [FB \oplus 02, FB, FB, AA] = [F9, FB, FB, AA]$

Words:

- $W_8 = W_4 \oplus g(W_7) = [C2, 63, 63, 63] \oplus [F9, FB, FB, AA]$
 $= [9B, 98, 98, C9]$

- $W_9 = W_5 \oplus W_8 = [F9, FB, FB, AA]$

- $W_{10} = W_6 \oplus W_9 = [9B, 98, 98, C9]$

- $W_{11} = W_7 \oplus W_{10} = [F9, FB, FB, AA]$

$$\therefore K_2 = \begin{bmatrix} 9B & F9 & 9B & F9 \\ 98 & FB & 98 & FB \\ 98 & FB & 98 & FB \\ C9 & AA & C9 & AA \end{bmatrix}$$

Time (20:43)

1.2) Polynomial Formation:

Each 4-byte column is treated as a polynomial $p(x)$ of degree < 4 with coefficients in $GF(2^8)/(x^8 + x^4 + x^3 + x + 1)$. The transformation computes,

$$p'(x) = c(x) \cdot p(x) \pmod{x^8 + 1},$$

$$\text{where } c(x) = 03x^3 + 01x^2 + 01x + 02$$

In matrix form:

$$\begin{bmatrix} p'_0 \\ p'_1 \\ p'_2 \\ p'_3 \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ p_3 \end{bmatrix}$$

Rule: Addition is $\oplus$. Multiplication: $02 \cdot b = x \text{ time}(b)$; $03 \cdot b = x \text{ time}(b) \oplus b$.

Step-by-step Trace on $p = [01, 02, 03, 04]^T$

Pre-multiplications:

- $02 \cdot 01 = 02$, $03 \cdot 01 = 03$
- $02 \cdot 02 = 04$, $03 \cdot 02 = 06$
- $02 \cdot 03 = 06$, $03 \cdot 03 = 06 \oplus 03 = 05$
- $02 \cdot 04 = 08$, $03 \cdot 04 = 08 \oplus 04 = 0C$

Byte Outputs:

- $p'_0 = 02 \oplus 06 \oplus 03 \oplus 09 = 03$
- $p'_1 = 01 \oplus 04 \oplus 05 \oplus 04 = 04$
- $p'_2 = 01 \oplus 02 \oplus 06 \oplus 0C = 09$
- $p'_3 = 03 \oplus 02 \oplus 03 \oplus 08 = 0A$

$$\therefore S' = [0 \times 03, 0 \times 04, 0 \times 09, 0 \times 0A]^T \quad (17:35)$$

20. Side-Channel Vulnerabilities:

- **Cache-Timing Attacks:** Table-based S-boxes use key-dependent indices ($P_i \oplus K_i$). Cache hits vs misses cause measurable execution time variations, allowing attackers (e.g., via Flush + Reload) to deduce memory addresses and extract key bytes.
- **Differential Power Analysis (DPA):** CMOS switching power correlates with Hamming weight/distance of intermediate states (e.g., $S(P_i \oplus K_i)$). Statistical analysis of power traces across multiple runs isolates the secret key.

Countermeasures & Trade-offs:

Masking (DPA Defense):

- **Mechanism:** Blinds sensitive data with random shares ($x = x' \oplus r$). Linear steps run directly; S-boxes use secure multi-party logic so power consumption never correlates with the actual key.
- **Trade-off:** 2X-5X performance slowdown, code complexity and requires a high throughput True Random Number Generator (TRNG).

Constant-Time S-Box / Hardware Instructions:

- Mechanism: Replaces lookup tables with bitlicing (Pure boolean AND/XOR in composite fields) or CPU hardware instructions (AES-NI) that execute in fixed clock cycles independent of cache states.
- Trade-off: Bitlicing has higher single-block latency and needs batch parallelization; AES-NI requires dedicated hardware (silicon support).

(13:15)

□ RSA Signatures, IPsec & Diffie-Hellman:

21 | RSA Procedures (via Euler's Totient Theorem):

Key Generation:

- Modulus: $n = pq$ (p, q distinct primes);
Totient: $\phi(n) = (p-1)(q-1)$
- Select e such that $\gcd(e, \phi(n)) = 1$
- Compute d such that $e \cdot d \equiv 1 \pmod{\phi(n)}$
 $\Rightarrow ed = 1 + k(p-1)(q-1)$
- Public Key: (e, n) | Private Key: (d, n)

Encryption & Decryption:

- $C \equiv M^e \pmod{n} \Rightarrow M' \equiv C^d \equiv M^{ed} \pmod{n}$.

Correctness Proof (General case: $m^{ed} \equiv m \pmod{n}$):

By CRT, $m^{ed} \equiv m \pmod{pq}$ holds if and only if it holds modulo p and modulo q .

Modulo p :

- If $p \nmid m$ ($\gcd(m, p) = 1$): By Fermat's little Theorem ($m^{p-1} \equiv 1 \pmod{p}$):

$$m^{ed} = m \cdot (m^{p-1})^{k(q-1)} \equiv m \cdot (1)^{k(q-1)} \equiv m \pmod{p}$$

- If $p \mid m$ ($\gcd(m, p) \neq 1$): $m \equiv 0 \pmod{p}$, so:

$$m^{ed} \equiv 0^{ed} \equiv 0 \equiv m \pmod{p}$$

Thus, $m^{ed} \equiv m \pmod{p}$ holds for all m .

- Modulo q : By identical symmetry, $m^{ed} \equiv m \pmod{q}$ holds for all m .

$$p \mid (m^{ed} - m) \text{ and } q \mid (m^{ed} - m)$$

Since $\gcd(p, q) = 1$, their product divides the difference:

$$pq \mid (m^{ed} - m) \Rightarrow m^{ed} \equiv m \pmod{n}$$

Holds for all $0 \leq m < n$, including when $\gcd(m, n) \neq 1$.

(12:55)

2.4 Key Pair Generation:

Modulus (n): $n = p \times q = 11 \times 13 = 143$

Totient ($\phi(n)$): $\phi(n) = (p-1)(q-1) = 10 \times 12 = 120$

Public Exponent (e): Choose smallest valid integer $e > 1$ coprime to 120:

$$\gcd(7, 120) = 1 \Rightarrow e = 7$$

Private Exponent (d): Compute $d \equiv e^{-1} \pmod{\phi(n)}$ via Extended Euclidean Algorithm ($7d \equiv 1 \pmod{120}$):

$$120 = 17 \times 7 + 1 \Rightarrow 1 = 120 - 17(7)$$

$$\Rightarrow 7 \times (-17) \equiv 1 \pmod{120}$$

$$d \equiv -17 \equiv 120 - 17 = 103$$

Keys:

Public Key: $(e=7, n=143)$ | Private Key: $(d=103, n=143)$

Signature Generation ($S \equiv H(m)^d \pmod{n}$):

Given, $H(m) = 23$:

$$S \equiv 23^{103} \pmod{143}$$

Modular Exponentiation by Repeated Squaring
($103 = 64 + 32 + 4 + 2 + 1$):

$$23^1 \equiv 23 \pmod{143}$$

$$23^2 \equiv 529 \equiv 100 \pmod{143}$$

$$23^4 \equiv 100^2 \equiv 10,000 \equiv 133 \equiv -10 \pmod{143}$$

$$23^8 \equiv (-10)^2 \equiv 100 \pmod{143}$$

$$23^{16} \equiv (100)^2 \equiv 133 \pmod{143}$$

$$23^{32} \equiv 100 \pmod{143}$$

$$23^{64} \equiv 133 \pmod{143}$$

Combine Components:

$$S \equiv 23^{64} \times 23^{32} \times 23^9 \times 23^2 \times 23^1 \pmod{143}$$

$$S \equiv (133 \times 100) \times (133 \times 100) \times 23 \pmod{143}$$

$$\text{Here, } 133 \times 100 \equiv (-10) \times 100 = -1000 \equiv -1 \pmod{143}$$

$$\therefore S \equiv (-1) \times (-1) \times 23 \equiv 23$$

$\therefore$ Digital Signature, $S = 23$

Signature Verification ($V \equiv S^e \pmod{n}$):

Verify that $V \equiv S^e \equiv H(m) \pmod{143}$:

$$V \equiv 23^7 \pmod{143}$$

Exponent Decomposition ($7 = 4 + 2 + 1$):

$$V \equiv 23^4 \times 23^2 \times 23^1 \pmod{143}$$

$$V \equiv 133 \times 100 \times 23 \pmod{143}$$

$$V \equiv (-10 \times 100) \times 23 \equiv (-1000) \times 23 \pmod{143}$$

Since $-1000 \equiv 1 \pmod{143}$:

$$V \equiv 1 \times 23 \equiv 23$$

Since $V = 23 = H(m)$, the signature is valid and verified.

(15:19)

23/ General Formula for IPsec ESP Tunnel Mode overhead:

In ESP Tunnel mode, the original IP packet (Payload L_{payload}) is completely encapsulated inside a new outer IP packet:

$$\text{Total Packet size } (L_{\text{total}}) = L_{\text{payload}} + L_{\text{overhead}}$$

Where the total overhead L_{overhead} consists of:

$$L_{\text{overhead}} = H_{\text{outer IP}} + H_{\text{ESP}} + IV + L_{\text{pad}} + T_{\text{ESP}} + ICV$$

$H_{\text{outer IP}}$: Outer IP header (typically 20 bytes for IPv4)

H_{ESP} : ESP header (8 bytes: SPI 4B + Sequence Number 4B)

IV : Initialization Vector / Nonce (e.g. 8-16 bytes for CBC/GCM; 0B if included in header spec)

L_{pad} : ESP padding (0-255 bytes)

T_{ESP} : ESP Trailer fields (2 bytes: Pad length 1B + Next Header 1B)

ICV : Authentication Data / Integrity check value

Overhead Percentage (%) = $\frac{L_{\text{overhead}}}{L_{\text{payload}}} \times 100\%$

$$= \frac{L_{\text{overhead}}}{L_{\text{payload}}} \times 100\%$$

Numerical Evaluation:

Given Parameters:

$$\text{Payload, } L_{\text{payload}} = 1200 \text{ bytes}$$

$$\text{ESP Header, } H_{\text{ESP}} = 8 \text{ bytes}$$

$$\text{Authentication Data, } I_{\text{CV}} = 16 \text{ bytes}$$

$$\text{Padding, } L_{\text{pad}} = 2 \text{ bytes}$$

$$\text{Outer IPv4 Header, } H_{\text{outerIP}} = 20 \text{ bytes}$$

$$\text{ESP Trailer (T}_{\text{ESP}}: \text{Pad Length} + \text{Next Header}) = 2 \text{ bytes}$$

$$\therefore \text{Fixed Overhead, } L_{\text{overhead}} = 20 + 8 + 2 + 2 + 16 = 48 \text{ bytes}$$

Overhead relative to payload:

$$\text{Overhead \%} = \frac{48}{1200} \times 100\% = 4.0\%$$

$$\text{Relative to total wire size: } \frac{48}{1200 + 48} \times 100\% \approx 3.85\%$$

Scaling: Small IoT Packets v. Large Payloads:

Small IoT Payloads (e.g., $L_{\text{payload}} = 20-50$ bytes):

- Overhead is disproportionately catastrophic:

$$\frac{48 \text{ bytes}}{20 \text{ bytes}} \times 100\% = 240\%$$

- Fixed encapsulation headers consume 2-3 x more bandwidth and radio transmission power than the telemetry data itself, leading to severe battery drain and MTU fragmentation.

Large Payloads (e.g., $L_{\text{payload}} \geq 1200-1400$ bytes):

- The fixed overhead amortizes down to $< 4\%$, providing near-optimal bandwidth efficiency for bulk transfers. (32:11)

3.1 Numerical DH Trace ($P=23, g=5$):

Public keys:

- Aleya ($a=6$): $A = 5^6 \equiv (5^2)^3 \equiv 2^3 \equiv 8 \pmod{23}$
- Badol ($b=15$): $B = 5^{15} \equiv (5^{12}) \cdot 5^3 \equiv 2^6 \cdot 10 \equiv 64 \cdot 10 \equiv 18 \cdot 10 \equiv 180 \equiv 19 \pmod{23}$

Shared key:

- Aleya: $K_A = B^a \equiv 19^6 \equiv (-4)^6 \equiv 16^3 \equiv (-7)^3 \equiv -343 \equiv 2 \pmod{23}$
 - Badol: $K_B = A^b = 8^{15} \equiv 2^{45} \equiv (2^{22})^2 \cdot 2^1 \equiv 1^2 \cdot 2 \equiv 2 \pmod{23}$
- $\therefore K_A = K_B = 2$ (confirmed)

Mitm Vulnerability of Unauthenticated DH:

- Standard DH lacks entity authentication.
- An active attacker (Mallory) intercepts A and B , injects her own public value $M = g^m$ and establishes two separate keys: $k_1 = g^{am}$ with Aleya and $k_2 = g^{bm}$ with Badol.
- Mallory transparently decrypts, reads/modifies, and re-encrypts all subsequent traffic undetected.

Mitigation via Authenticated DH (STS Protocol):

- Mechanism: Uses public-key digital signatures to sign the exchanged parameters (e.g., Bob sends B along with $\text{Sig}_B(B, A)$ verified via Bob's public certificate).
- Defense: Mallory cannot forge valid signatures on her replaced parameters without the legitimate private signing keys, preventing key replacement. (17:15)

35 Computational Cost Comparison (128-bit security):

Scheme	Client Cost	Server Cost	Key/overhead
RSA Key Transport (3072-bit)	Very low ($M^2 \pmod N$, $e = 65537$)	High ($C^d \pmod n$, 3072-bit CRT decrypt)	384B
Classic DHE (3072-bit)	High (2×3072 bit modular exp)	High (2×3072 bit modular exp)	384B
ECDHE (256 bit, e.g. x25519)	Very low (2×256 bit scalar mult)	Very low (2×256 bit scalar mult)	32B

- ECDHE executes 5X-10X faster than RSA-3072 decryption and > 20 X faster than 3072-bit DHE with 1/12th the key size.

Forward Secrecy: why Modern TLS Favors (EC)DHE:

- RSA Key Transport: Client encrypts premaster secret with server's static public key. If the private key is ever leaked later, an adversary who recorded past network traffic can decrypt all historical sessions.
- (EC)DHE: Uses fresh, ephemeral keypair (a, g^a) discarded immediately after handshake. The server's static key only signs the exchange. Leaking the private key later compromises no past traffic. Hence, TLS 1.3 completely dropped RSA key exchange. (11: 49)

ElGamal Signature & Elliptic Curve Cryptography

26 | Public Key Computation(y):

Given, $P = 467$, $g = 2$, Private Key, $x = 127$:

$$y = g^x \pmod{P} = 2^{127} \pmod{467}$$

$$y = 132$$

$\therefore$ Public key: $(P = 467, g = 2, y = 132)$

Signature Generation(a, s):

Given $H(M) = 100$, ephemeral key, $k = 213$ and modulus for exponent arithmetic $P-1 = 466$:

Compute r :

$$r = g^k \pmod{p} = 2^{213} \pmod{467} = 29$$

Compute $k^{-1} \pmod{p-1}$ via Extended Euclidean Algorithm:

$$\text{gcd}(213, 466):$$

$$466 = 2(213) + 40$$

$$213 = 5(40) + 13$$

$$40 = 3(13) + 1 \Rightarrow 1 = 40 - 3(13)$$

Substituting back:

$$\begin{aligned} 1 &= 16(40) - 3(213) = 16(466 - 2 \cdot 213) - 3(213) = 16(466) - 35(213) \\ &= 16(466) - 35(213) \end{aligned}$$

$$k^{-1} \equiv -35 \equiv 466 - 35 \equiv 431 \pmod{466}$$

Compute s :

$$s = k^{-1} \cdot [H(m) - x \cdot r] \pmod{p-1}$$

$$x \cdot r = 127 \times 29 = 3683 \equiv 421 \pmod{466}$$

$$H(m) - x \cdot r = 100 - 421 = -321 \equiv 145 \pmod{466}$$

$$s \equiv 431 \times 145 = 62495 \equiv 51 \pmod{466}$$

$$\therefore \text{Digital signature, } (r, s) = (29, 51)$$

Signature verification:

$$\text{Verify that } g^{H(m)} \equiv y^r \cdot r^s \pmod{p}:$$

$$\text{L.H.S.} \Rightarrow V_1 = g^{H(m)} \pmod{p} = 2^{100} \pmod{467} = 189$$

R.H.S $\Rightarrow$

$$v_2 = y^r \cdot r^s \pmod{p} = (132^{29} \times 29^{51}) \pmod{467}$$

$$132^{29} \equiv 396 \pmod{467}$$

$$29^{51} \equiv 26 \pmod{467}$$

$$\therefore v_2 \equiv (396 \times 26) = 10296 \equiv 189 \pmod{467}$$

Since $v_1 = v_2 = 189$, the signature is valid and verified. (19 : 33)

23/ Definitions

Public key, $y \equiv g^x \pmod{p}$:

Signature Components, $r \equiv g^k \pmod{p}$

$$s \equiv k^{-1}(H(m) - xr) \pmod{p-1}$$

Algebraic Proof:

Multiply by k :

$$kr = (H(m) - xr) \pmod{p-1}$$

Rearrange for $H(m)$:

$$H(m) = xr + kr + t(p-1)$$

Exponentiate base g modulo p :

$$g^{H(m)} \equiv g^{xr + kr + t(p-1)} \pmod{p}$$

$$g^{H(m)} \equiv (g^x)^r \cdot (g^k)^t \cdot (g^{p-1})^t \pmod{p}$$

By Fermat's Little Theorem,

$$g^{p-1} \equiv 1 \pmod{p} \Rightarrow (g^{p-1})^t \equiv 1 :$$

$$g^{H(m)} \equiv (g^x)^a \cdot (g^k)^n \cdot 1 \pmod{p}$$

Substitute $y = g^x$ and $r = g^k$:

$$g^{H(m)} \equiv y^a \cdot r^n \pmod{p}$$

The verification equation holds. (7:53)

28 Curve Equation, $E: y^2 \equiv x^3 + x + 1 \pmod{23}$ ($a=1, b=1$)

Verification of $P = (3, 10)$:

$$L.H.S \equiv y^2 = 10^2 = 100 \equiv 40 \times 23 + 8 \equiv 8 \pmod{23}$$

$$R.H.S \equiv x^3 + x + 1 = 3^3 + 3 + 1 = 27 + 4 \equiv 31 \equiv 8 \pmod{23}$$

$$L.H.S = R.H.S \Rightarrow P(3, 10) \text{ lies on the curve.}$$

Point Doubling: $2P = (x_3, y_3)$ for $P(3, 10)$:

Slope (λ):

$$\begin{aligned} \lambda &= \frac{3x_1^2 + a}{2y_1} \pmod{23} = \frac{3(3^2) + 1}{2(10)} = \frac{28}{20} \equiv \frac{5}{-3} \\ &\equiv \frac{5}{20} \pmod{23} \end{aligned}$$

• Dominator inverse: $20^{-1} \equiv (-3)^{-1} \pmod{23}$

$$\text{Since } (-3) \times (-8) = 24 \equiv 1 \pmod{23};$$

$$20^{-1} \equiv -8 \equiv 15 \pmod{23}$$

$$\lambda \equiv 5 \times 15 = 75 \equiv 3 \times 23 + 6 \equiv 6 \pmod{23}$$

• Coordinates of $2P$:

$$x_3 = \lambda^2 - 2x_1 = 6^2 - 2(3) = 36 - 6 = 30 \equiv 7 \pmod{23}$$

$$\begin{aligned} y_3 &= \lambda(x_1 - x_3) - y_1 = 6(3 - 7) - 10 = 6(-4) - 10 = -34 \\ &\equiv 46 - 34 = 12 \pmod{23} \end{aligned}$$

$$\therefore 2P = (7, 12)$$

Point Addition: $P(3, 10) + Q(9, 7) = (x_4, y_4)$:

Slope (λ'):

$$\lambda' = \frac{y_2 - y_1}{x_2 - x_1} = \frac{7 - 10}{9 - 3} = \frac{-3}{6} \pmod{23}$$

- Reduce fraction: $\frac{-3}{6} = \frac{-1}{2} \pmod{23}$

- Denominator inverse: $2^{-1} \equiv 12 \pmod{23}$ ($\because 2 \times 12 = 24 \equiv 1$)

$$\lambda' \equiv -1 \times 12 = -12 \equiv 23 - 12 = 11 \pmod{23}$$

Coordinate of $P+Q$:

$$x_4 = \lambda'^2 - x_1 - x_2 = 11^2 - 3 - 9 = 109 \equiv 17 \pmod{23}$$

($\because 4 \times 23 = 92$)

$$y_4 = \lambda'(x_1 - x_4) - y_1 = 11(3 - 17) - 10 = -164 \pmod{23}$$

$$-164 \equiv 8 \times 23 - 164 = 20 \pmod{23}$$

$$P+Q = (17, 20) \quad (17: 13)$$

29/ Given,

Curve $y^2 \equiv x^3 + 7x + 10 \pmod{37}$, $k = (2, 5)$, $n = 19$,
 $d = 9$, $k' = 3$, $H(m) = 8$.

Public key $R = 9k$:

$$\therefore 2k : \lambda = \frac{3(2^2) + 7}{2(5)} = \frac{19}{10} \equiv 19 \cdot 26 \equiv 13 \pmod{37}$$

$$x = 13^2 - 4 \equiv 17, y = 13(2 - 17) - 5 \equiv 22 \quad \Rightarrow$$

$$\Rightarrow 2k = (17, 22)$$

$$\bullet 3h = 2h + h \neq 1; \lambda = \frac{22-5}{17-2} = \frac{17}{15} \equiv 17 \cdot 5 \equiv 11 \pmod{37}$$

$$x = 11^2 - 19 \equiv 28, y = (11(17-28) - 22) \equiv 5$$

$$\Rightarrow 3h = (28, 5)$$

$\bullet 9h$: Doubling $4h = (7, 32)$, $8h = (13, 16)$ and adding h yields:

$$Q = 9h = (23, 11)$$

Signature Generation (r, s) :

$\bullet$ Ephemeral point: $R = kh = 3h = (28, 5)$

$$\bullet r: r = x_R \pmod{n} = 28 \pmod{19} = 9$$

$$\bullet s: s = k^{-1}(H(M) + d \cdot 1) \pmod{n}$$

$$\bullet k^{-1} = 3^{-1} \equiv 13 \pmod{19}$$

$$\bullet s \equiv 13 \cdot (8 + 9 \cdot 2) = 13 \cdot (8 + 5) = 13 \cdot 13 = 169 \equiv 17 \pmod{19}$$

$$\therefore \text{Signature } s, (r, s) = (9, 17)$$

Verification:

Weights $(\pmod{19})$:

$$\bullet w = s^{-1} = 17^{-1} \equiv 9 \pmod{19}$$

$$\bullet u_1 = H(M) \cdot w = 8 \cdot 9 = 72 \equiv 15 \pmod{19}$$

$$\bullet u_2 = r \cdot w = 9 \cdot 9 = 81 \equiv 5 \pmod{19}$$

Point Recovery:

$$P = u_1 h + u_2 Q = 15h + 5(9h) = 60h \equiv (60 \pmod{19})h$$

$$P = 3h = (28, 5)$$

$$\text{Check: } V = x_P \pmod{n} = 28 \pmod{19} = 9 = r$$

$\Rightarrow$ Verified.

(10:15)

30. Key-Size security Equivalence:

Symmetric security	RSA/ElGamal key size	ECC key size	Ratio
128-bit	3072-bit	256-bit	9:1
128-bit (Standard)	3072-bit	256-bit	12:1
256-bit (High)	15360-bit	512-bit	30:1

Mathematical Reason: RSA and ElGamal face sub-exponential attacks (GNFS/Index Calculus), forcing massive key scaling. ECC faces only fully exponential attacks (Pollard's rho, $\Theta(\sqrt{n})$), achieving equivalent security with tiny keys.

Why ECC is preferred for smart cards & IoT:

- **Minimal Storage:** 256-bit ECC keys require only 32 bytes, saving scarce EEPROM/ROM.
- **Faster Computation:** Scalar Multiplication on 256-bit curves runs 5x-10x faster than 3072-bit modular exponentiation.
- **Low power Consumption:** Shorter CPU cycles drastically cut battery drain on remote sensor nodes.
- **Bandwidth Efficiency:** Compact keys/signatures easily fit within low-power packet sizes without MTU fragmentation.

(10:00)

Hash Functions & Lattice-Based Cryptography

3.1 Definitions:

- **Preimage Resistance:** Given y , finding any x such that $H(x) = y$ is infeasible (Cost: $O(2^n)$).
- **Second-Preimage Resistance:** Given x_1 , finding $x_2 \neq x_1$ such that $H(x_2) = H(x_1)$ is infeasible (Cost: $O(2^n)$).
- **Collision Resistance:** Finding any pair $x_1 \neq x_2$ such that $H(x_1) = H(x_2)$ is infeasible (Cost: $O(2^{n/2})$).

Birthday-Attack Derivation ($\approx 2^{n/2}$):

for $N = 2^n$ possible hash values and k sampled messages:

1. Probability of no collision:

$$P_{\text{no col}} = \prod_{i=1}^{k-1} \left(1 - \frac{i}{N}\right) \approx \prod_{i=1}^{k-1} e^{-i/N} = \exp\left(-\sum_{i=1}^{k-1} \frac{i}{N}\right) \\ \approx e^{-k^2/(2N)}$$

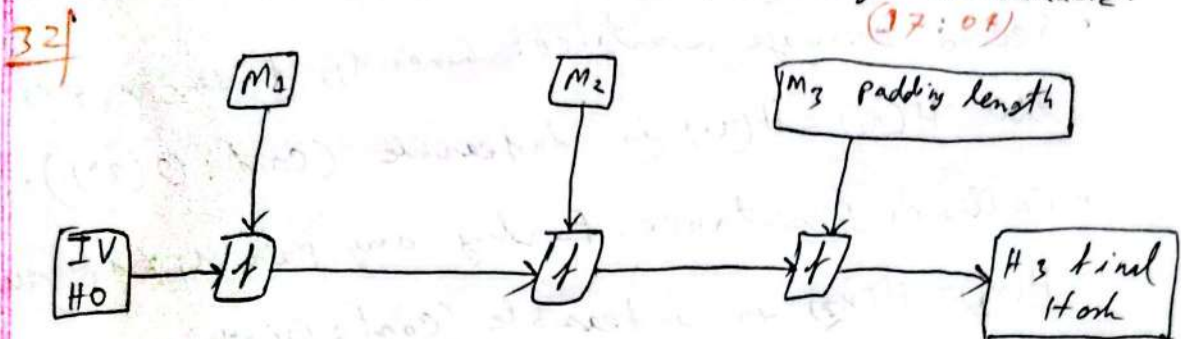
2. 50% Collision condition ($P_{\text{col}} = 1 - P_{\text{no col}} \geq 0.5$):

$$e^{-k^2/(2N)} \leq \frac{1}{2} \Rightarrow \frac{k^2}{2N} \geq \ln 2 \Rightarrow k \approx \sqrt{2 \ln 2 \cdot N} \\ \approx 1.177 \sqrt{2^n} \approx 2^{n/2}$$

Why 256-Bit Over 128-Bit:

- 128-bit hash (e.g., MD5): Collision search takes only $2^{128/2} = 2^{64}$ operations - easily broken today by cloud GPU, to forge digital signatures.

- 256-bit hash (e.g., SHA-256): Collision search requires $2^{256/2} = 2^{128}$ operations, providing standard 128-bit security which is practically unbreakable. (17:01)



The length-Extension Attack:

The flaw: The published hash output is the entire internal state. So if an attacker knows $H(M)$ and the length of M , they can set $H_0' = H(M)$ and continue hashing additional blocks.

Computing $H(M || \text{padding} || M')$ without ever knowing M itself.

How HMAC Avoid it: HMAC uses nested hashing with two derived keys:

$$\text{HMAC}(K, m) = H((K \oplus \text{Opad}) || ((K \oplus \text{ipad}) || m))$$

The inner hash output is rehashed with the outer key. An attacker extending the inner hash cannot compute the outer hash without knowing K , so the extension

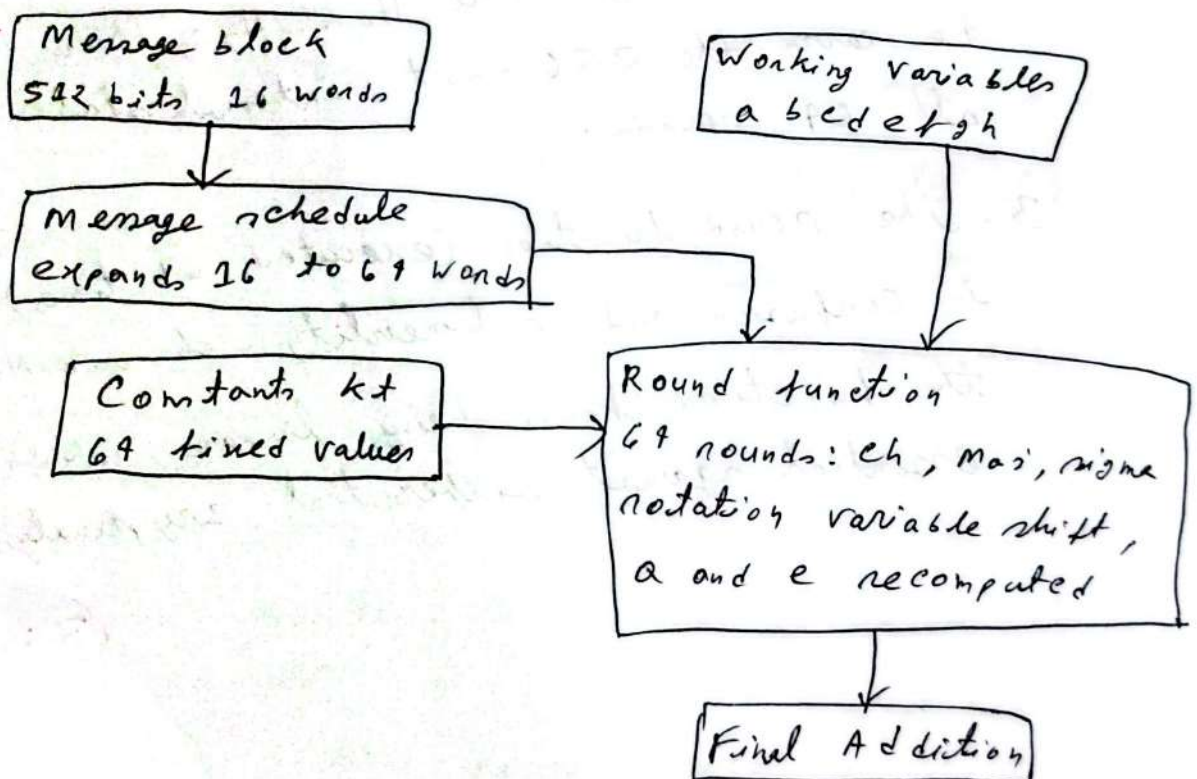
never produces a valid tag.

How SHA-3 (Sponge Construction) Avoids it:

SHA-3 keeps an internal state of $b = r + c$ bits, split into a rate r and a capacity c . Absorbing XORs message blocks into the rate portion only; squeezing output bits from the rate portion only.

(10:12)

33



Role of each stage:

1. Message schedule expansion: The 512-bit input blocks are first split into sixteen 32-bit words $w_0, \dots, w_{15}$. These are then expanded to sixty-four words $w_0, \dots, w_{63}$ using the recurrence.
2. The eight working variables (a, b, c, d, e, f, g, h): These 32-bit registers are initialized from the previous chaining value $H_0, \dots, H_7$. Their role is to carry the 256-bit internal state through all 64 rounds.
3. The round function (executed 64 times): Its role is confusion and non-linearity - χ and Maj prevent the function from being linear, which is essential against differential cryptanalysis.

(7:14)

34. Lattice: A discrete set of points $L(B) = \{ \sum z_i b_i : z_i \in \mathbb{Z} \}$ generated by integer combinations of basis vectors $b_1, \dots, b_J$.

Shortest vector problem (SVP): Given a lattice basis, find the non-zero vector $v \in L$ with the smallest length. Where $\lambda_1(L)$ is the length of the shortest non-zero vector in the lattice.

Shortest Independent vectors problem: Given a basis of an n -dimensional lattice, find n linearly independent vectors $v_1, \dots, v_n \in L$ such that the maximum length among them is at most $\lambda_n(L)$.

Why both resist quantum attacks:

- Shor's algorithm only works on abelian hidden subgroup problems - this is why it breaks factoring and discrete logs. SVP/SIVP instead reduce to the non-abelian dihedral HSP, for which no efficient quantum algorithm is known.
- Lattices have no algebraic structure for a Quantum Fourier Transformation to exploit.

- Worst-case to average case reduction:

Breaking a random lattice instance is as hard as solving SIVP in the worst case - giving a security guarantee RSA/ECC hack.

(12:37)

35/

Kyber's security is based on Module-LWE, a public matrix A and secret s produce $t = As + e \pmod{2}$, where e is a small random "noise" vector.

Module-LWE: uses small matrices of polynomials instead of one giant matrix or a single polynomial.

How noise hides the secret: Since e is the small but random, t looks statistically indistinguishable from a uniformly random vector.

KEM flow: Public key = sender picks small random r, e_1, e_2 , sends ciphertext ($u = A^T r + e_1, v = t^T r + e_2 + \text{Encode}(m)$). Receiver computes $v - t^T u \approx \text{Encode}(m) + \text{small noise}$.

Why it resists classical attacks: Best known attacks run in exponential time in the lattice dimension; Kyber's parameters push this cost above 2^{128} operations.

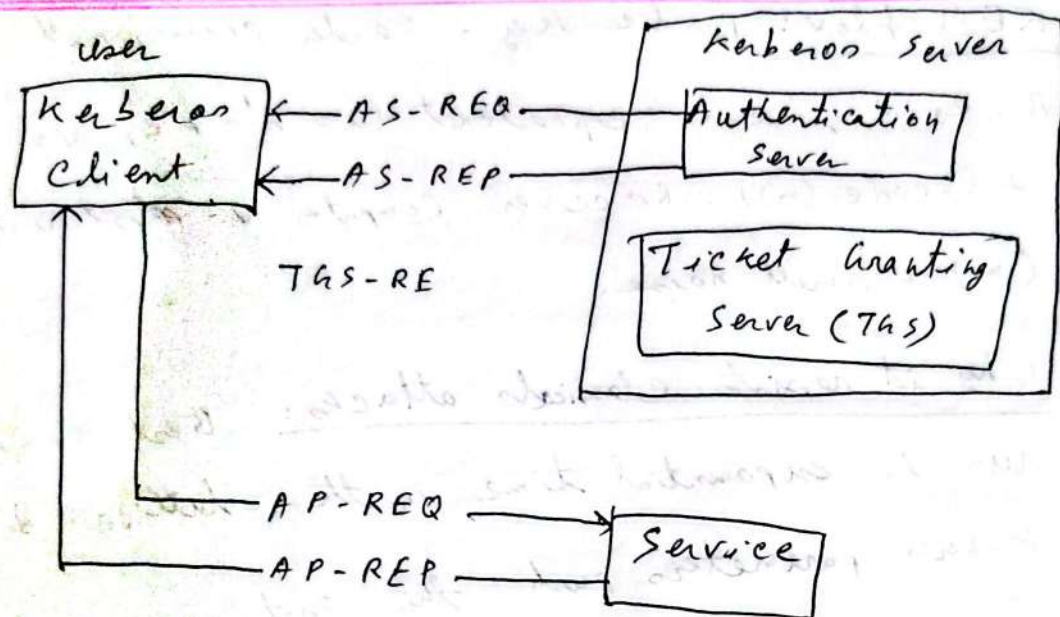
Why it resists quantum attacks: Module-LWE has no abelian periodic structure for Shor's algorithm to exploit. (9:15)

34

Applied Protocol Security: Kerberos, MAC, TLS & IoT

36

Kerberos uses a trusted authentication server and ticket granting server so that user's password is not repeatedly ret.



1. AS exchange:

Client sends its identity ID_c and requests a TGT. A_S generates a session key K_c , t_{g_s} and sends:

$$\{K_c, t_{g_s}, TGT\}_{K_c}$$

where K_c is derived from the user's password.

The TGT contains:

$$TGT = \{ID_c, \text{life time}, K_c, t_{g_s}\}_{K_{TGS}}$$

2. TGS exchange:

Client sends TGT and Authenticator.

$$\text{Authenticator} = \{ID_c, \text{timestamp}\}_{K_c, t_{g_s}}$$

TGS verifies and creates a new session key K_c, s

It returns:

$\{k_{cs}, \text{lifetime}\} k_{cs} + \text{Tickets}$

3. Client server exchange:

Client sends ticket and a new authenticator to the service server.

The server decrypts the ticket, obtains k_{cs} , and verifies the authenticator. This provides mutual authentication.

Why TGT avoids repeated password transmission:

The password is never sent on the network. It is used to obtain k_e and decrypt the AS response. After that, the client uses the TGT and temporary session keys to obtain service tickets.

(17:42)

32.

Why $H(k||M)$ is Vulnerable:

In Merkle-Damgård hashes, the output is the full internal state. So given a valid pair $(M, T = H(k||M))$, an attacker can resume hashing from T and append any suffix M' , producing a valid tag for $k||M||\text{padding}||M'$ - without knowing k .

How HMAC fixes it:

$$\text{HMAC}(k, m) = H((k \oplus \text{opad}) || (H((k \oplus \text{ipad}) || m)))$$

The inner hash result is never published directly. It's fed into a second, outer hash keyed with a different derived key. To extend the tag, an attacker would need to compute the outer hash, which requires knowing $k \oplus \text{opad}$. Since that's secret, extension attacks fail.

Try numeric example: Using toy hash $f(h, m) = (h + m) \times 7 \bmod 256$, $IV = 5$, $K = 12$, $M = [30, 90]$:

Naive MAC : $H(12 || 30 || 40) = 157$. Attacker
extends with 50 : $(157 + 50) \times 7 \bmod 256 = 169$
 $\Rightarrow$ accepted as valid MAC for $[30, 40, 50]$ without
knowing $k \times$ (broken)

HMAC : inner $BI = H(15 || 30 || 40) = 162$ (using
 $k \oplus \text{ipad} = 15$); outer $T = H(5 || 162) = 88$

Extending 88 with 50 gives 198 but correct
HMAC for $[30, 40, 50]$ requires redoing both inner
and outer hashing with the secret key, giving a
different value.
(15 : 33)

38

Flow:

- Client Hello (+ key - share guess) $\rightarrow$ Server Hello (+ key - share)
- $\rightarrow$ [Everything after this point is encrypted]
- $\rightarrow$ Encrypted Extensions, Certificate, certificate
Verify, Finished (server) $\rightarrow$ Finished (client)
- $\rightarrow$ Application data

Key derivation: Both sides compute the (EC) DHE shared secret right after ServerHello, then run it through HKDF-Extract/Expand chained with an early secret and handshake transcript hash to derive handshake traffic secrets and application traffic secrets.

Comparison:

	TLS 1.2	TLS 1.3
Round trips	2-RTT	1-RTT
Forward Secrecy	Optional (RSA key transport allowed)	Mandatory (EC) DHE only
Handshake Encryption	Certs sent in plaintext	Encrypted after ServerHello

TLS 1.2 needs 2 round trips because the client must wait for the server's key exchange parameters before sending its own. TLS 1.3 removes this by letting the client guess and send a key share immediately, cutting to 1-RTT.

(10:18)

39 | Data volume per node/day: $1200 \text{ bytes} \times 1440$
(Packets/day) = $1728,000 \text{ bytes} = 1687.5 \text{ KB/day}$.

Airtime issue (LoRaWAN, SF9): Max payload per
frame $\approx 115 \text{ bytes}$, so 1200 bytes needs ~ 11
fragmented frames, each $\sim 0.33 \text{ s}$ airtime $\rightarrow \sim 3.63 \text{ s}$
per transmission $\times 60/\text{hour} = \sim 218 \text{ s/hour}$ of airtime.
Regulatory limit: EU 1% duty cycle allows only
 36 s/hour .

Verify the Signature:

The Verifier calculates, $H' = s^e \bmod n$
 $= 23^7 \bmod 143$

$$\text{Now, } 23^2 \equiv 100$$

$$23^4 \equiv 133$$

$$\text{Therefore, } 23^7 = 23^4 \times 23^2 \times 23$$

$$\equiv 133 \times 100 \times 23 \pmod{143}$$

$$\therefore H' = 23$$

Since $H' = H(M) = 23$

the signature is valid.

(5: 19)

40/

Criterion	Ancon-128	AES-128
Memory	~1-2KB Code, no lookup tables	Larger AES-GCM needs multiplication tables (256B - 4KB)
Energy/byte	Lower (Simple bitwise permutation)	Higher (Mixcolumns + GHASH multiplication)
Side-channel resistance	Naturally constant-time (small bitblended S-box), cheap to mask	Table-based lookups leak via cache-timing; masking is expensive
Maturity	New (NIST 2023 standard)	20+ years analysis, hardware acceleration common.

Recommendation: For a battery-powered sensor with <30KB RAM and no AES hardware acceleration, choose Ancon-128. It combines AEAD + hashing in one small, table-free, low-energy, naturally side-channel-resistant primitive. If the chip does have AES hardware acceleration, AES-128-GCM becomes the better choice instead.

(7:18)